CHAPTER 7 · TERRAFORM DEEP DIVE

Terraform Provisioners & Null Resources

Bootstrapping servers, running one-time scripts, and re-triggering actions safely with local-exec, remote-exec, file, and null_resource.

Instructor Deck

7.1

When to Use Provisioners?

Terraform is designed to **manage infrastructure, not configuration.** Provisioners run scripts or commands after a resource is created — use them only when necessary.

Use Provisioners When

- ✓ Installing software (e.g., nginx on an EC2 instance)
- ✓ Copying files to remote servers
- ✓ Running one-time scripts for setup

Avoid Provisioners When

- ✗ Config management tools are available (Ansible, Puppet, Chef)
- ✗ Cloud-init or user-data can achieve the same result

7.2

Types of Provisioners

1

`local-exec`

Runs commands on the machine running Terraform.

2

`remote-exec`

Runs commands inside the remote instance over SSH/WinRM.

3

`file`

Copies files from the local machine to the remote instance.

Each provisioner attaches to a resource block and runs once, at create (or destroy) time.

7.3

Using the local-exec Provisioner

Runs on the machine executing Terraform — not on the created resource.

- Logging or notifying (e.g., Slack) after apply
- Generating local files from resource output
- Triggering local CLI tools or scripts

Example use: notify Slack after an EC2 instance is created, logging its instance ID locally.

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  provisioner "local-exec" {
    command = "echo 'EC2 instance
      ${self.id} has been created'
      >> ec2-log.txt"
  }
}
```

7.4

Using the remote-exec Provisioner

Logs into the remote server (typically via SSH) and runs commands directly on it.

- Requires a connection block (SSH/WinRM)
- Needs network access + open security group
- Great for install & bootstrap commands

Example use: SSH into a fresh EC2 instance and install + start Nginx right after it's created.

```
connection {
  type        = "ssh"
  user        = "ubuntu"
  private_key = file("~/.ssh/my-key.pem")
  host        = self.public_ip
}

provisioner "remote-exec" {
  inline = [
    "sudo apt update -y",
    "sudo apt install nginx -y",
    "sudo systemctl start nginx"
  ]
}
```

7.5

Using the file Provisioner

Copies files or directories from the local machine to the newly created remote server.

- Requires the same connection block as remote-exec
- Often paired with remote-exec to run the copied file
- Good for config files, scripts, or small assets

Example use: push an nginx app-config file straight into /etc/nginx/conf.d/ on the instance.

```
connection {
  type        = "ssh"
  user        = "ubuntu"
  private_key = file("~/.ssh/my-key.pem")
  host        = self.public_ip
}

provisioner "file" {
  source      = "app-config.conf"
  destination =
    "/etc/nginx/conf.d/app-config.conf"
}
```

7.6

Using Null Resources

A **`null_resource`** does nothing on its own — it only exists to trigger provisioners when its inputs change.

`triggers { }`

The map of values under triggers is compared on every plan. When any value changes, the provisioner re-runs.

Common uses: re-run setup scripts, force dependency ordering, or bridge resources with no native relationship.

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "null_resource" "vpc_change" {
  triggers = {
    vpc_id = aws_vpc.main.id
  }

  provisioner "local-exec" {
    command = "echo 'VPC changed!'"
  }
}
```

If the VPC changes, Terraform runs the script again — even though nothing else about the resource changed.

Avoiding Anti-Patterns with Provisioners

Using provisioners for ongoing configuration management	→	Use Ansible, Chef, or Puppet for continuous config updates
Hardcoding sensitive credentials directly in scripts	→	Store secrets in Vault or AWS Secrets Manager
Running provisioners on every single terraform apply	→	Use null_resource triggers to run only when inputs change

WORKED EXAMPLE

Run a Shell Script on EC2 After Creation

Project Structure

```
terraform-project/
├── main.tf      # Terraform config
└── setup.sh     # Script for EC2
```

The flow: Terraform launches 20 EC2 instances, copies setup.sh to each with the file provisioner, then runs it with remote-exec.

setup.sh

```
#!/bin/bash
sudo apt update -y
sudo apt install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
```

WORKED EXAMPLE

Terraform Configuration (main.tf)

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  count         = 20
  key_name      = "my-key"

  connection {
    type        = "ssh"
    user        = "ubuntu"
    private_key = file("~/.ssh/my-key.pem")
    host        = self.public_ip
  }

  provisioner "file" {
    source      = "setup.sh"
    destination = "/home/ubuntu/setup.sh"
  }

  provisioner "remote-exec" {
```

Apply It

```
$ terraform init
$ terraform apply \
    -auto-approve
```

What Happens

- → Creates 20 EC2 instances
- → Copies setup.sh to each
- → Runs setup.sh via SSH
- → Apache installed & started

CHAPTER 7 RECAP

Summary

- What provisioners are and when to reach for them
- local-exec, remote-exec, and file provisioners
- null_resource for re-triggering scripts on change
- Anti-patterns to avoid — and their fixes
- A full worked example: bootstrap EC2 with Apache